

Criptografia Assimetrica

Q1 - Perfect Forward Secrecy (PFS)

A questao coloca se a aplicacao desenvolvida garante PFS (Perfect Forward Secrecy), assumindo que sao trocadas varias mensagens usando a KDF para derivar as chaves.

Resposta: Sim, o programa garante PFS

O PFS (Perfect Forward Secrecy) garante que, mesmo que o segredo K seja comprometido no futuro, as comunicacoes passadas continuam seguras. O programa garante esta propriedade pelos seguintes motivos:

1. Chaves efemerias por sessao:

Em cada execucao do programa, Alice e Bob geram novas chaves privadas DH (x e y) de forma independente. Estas chaves sao efemerias - existem apenas durante a sessao e sao descartadas no fim:

```
Sessao 1: x1, y1 -> K1 -> chave_aes_1
Sessao 2: x2, y2 -> K2 -> chave_aes_2
Sessao 3: x3, y3 -> K3 -> chave_aes_3
```

2. Independencia entre sessoes:

Cada sessao produz um segredo K diferente porque as chaves privadas x e y sao sempre novas. Um atacante que comprometa K2 apenas consegue decifrar as mensagens da sessao 2 - as sessoes 1 e 3 continuam protegidas porque as chaves privadas que geraram K1 e K3 ja foram descartadas e nao podem ser recuperadas.

3. Sem PFS seria assim:

```
Chave privada fixa x -> K sempre igual -> todas as sessoes
comprometidas!
```

Caracteristica	Com PFS (programa atual)	Sem PFS
Chaves DH	Novas por sessao (efemerias)	Fixas e reutilizadas
K comprometido	Afeta apenas 1 sessao	Afeta todas as sessoes
Sessoes passadas	Continuam seguras	Todas comprometidas

Conclusao: O programa garante PFS porque as chaves DH sao efemerias e geradas de novo em cada sessao, garantindo que o compromisso futuro de K nao afeta a seguranca das comunicacoes passadas.

Q2 - Localizacao das Chaves Publicas

As chaves publicas de cada participante estao armazenadas dentro dos proprios certificados X509, no campo Subject Public Key Info.

Inspecionando o certificado de Bob com o comando:

```
openssl x509 -in Bob.crt -text -noout
```

E possível observar a seguinte estrutura:

```
Subject: CN=Bob
Subject Public Key Info:
  Public Key Algorithm: rsaEncryption
    Public-Key: (2048 bit)
    Modulus: 00:d8:1a:ce:33:...  <- chave publica RSA de Bob
    Exponent: 65537
```

Cada certificado contém os seguintes campos relevantes:

Campo	Conteúdo
Subject	Identidade do dono (CN=Bob)
Subject Public Key Info	Chave pública RSA de 2048 bits
Issuer	Quem assinou o certificado (CN=CA)
Validity	Período de validade do certificado
Signature Value	Assinatura da CA que garante autenticidade

A CA assina o certificado com a sua chave privada. Qualquer participante que confie na CA pode verificar a assinatura com a chave pública da CA e confirmar que a chave pública no certificado pertence legitimamente ao seu titular.

Hierarquia de confiança:

```
CA (auto-assinada - raiz de confiança)
|-- Certificado de Alice (assinado pela CA)
|-- Certificado de Bob   (assinado pela CA)
```

Q3 - Verificação do Certificado no Protocolo STS

A questão coloca se o protocolo Station-to-Station continua imune a ataques Man-in-the-Middle se Alice omitir a verificação do certificado de Bob com a CA, mantendo apenas a verificação da assinatura.

Resposta: Não, o protocolo fica vulnerável a MitM

Se Alice não verificar o certificado de Bob com a CA, o ataque Man-in-the-Middle torna-se possível da seguinte forma:

Passo	Alice	Mallory	Bob
1	Envia gx	Intercepta gx, envia gm a Bob	Recebe gm
2	Recebe gm + SigM + CertM	Intercepta gy + SigB + CertB, envia gm + SigM + CertM a Alice	Envia gy + SigB + CertB
3	Verifica SigM com CertM - válido!	Encaminha para Bob	Recebe SigA + CertA
4	K _{am} = gxm	K _{am} com Alice, K _{mb} com Bob	K _{mb} = gmy

O ataque funciona porque:

- Alice verifica que a assinatura SigM é válida para o certificado CertM recebido - e de facto válida!
- Mas como Alice não verifica CertM com a CA, não sabe se CertM pertence mesmo ao Bob
- Mallory criou CertM com a sua própria chave pública e assinou com a sua chave privada
- Alice aceita CertM como se fosse o certificado do Bob - estabelece K_{am} com Mallory em vez de K com Bob

A verificação com a CA é essencial porque:

Com verificação da CA	Sem verificação da CA
CertM não foi assinado pela CA	Alice aceita qualquer certificado
CA não reconhece CertM - REJEITADO	Mallory pode usar o seu próprio CertM
Protocolo imune a MitM	Protocolo vulnerável a MitM

Conclusão: A verificação do certificado com a CA é uma componente fundamental do protocolo STS. Sem ela, a verificação da assinatura é inútil - um atacante pode sempre criar um certificado falso com a sua própria chave pública e assinar mensagens válidas com a sua chave privada. A CA é a raiz de confiança que garante que as chaves públicas pertencem mesmo a quem dizem pertencer.